

SMARTMOVELK: TRANSPORTATION GUIDANCE SYSTEM IN SRI LANKA

Seedin TM

248375F

MSC in Data Science and AI

Department of Computer Science and Engineering
Faculty of Engineering

University of Moratuwa
Sri Lanka

01 2024

SMARTMOVELK: TRANSPORTATION GUIDANCE SYSTEM IN SRI LANKA

Seedin TM

248375F

Dissertation submitted in partial fulfilment of the requirements for the
degree
MSC in Data Science and AI

Department of Computer Science and Engineering
Faculty of Engineering

University of Moratuwa
Sri Lanka

01 2024

DECLARATION

I declare that this is my own work and this Dissertation does not incorporate without acknowledgment any material previously submitted for a Degree or Diploma in any other University or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature:

Date:

The supervisor should certify the Dissertation with the following declaration.

The above candidate has carried out research for the MSC in Data Science and AI Dissertation under my supervision. I confirm that the declaration made above by the student is true and correct.

Name of Supervisor:

Signature of the Supervisor:

Date:

ACKNOWLEDGEMENT

I sincerely thank **Prof.Gihan Dias** for his invaluable guidance, continuous support, and supervision throughout this research. His insightful feedback and expertise have been instrumental in shaping this work. I also extend my gratitude to my research group for their valuable discussions and assistance, which greatly contributed to the progress of this study.

ABSTRACT

Abstract Text

Keywords: research keywords

TABLE OF CONTENTS

Declaration of the Candidate & Supervisor	i
Acknowledgement	ii
Abstract	iii
Table of Contents	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Research Gap	2
1.5 Research Objectives	2
2 Literature Review	3
2.1 Overview	3
2.2 Transportation Information Systems	3
2.3 Intelligent Transportation Systems	4
2.4 Knowledge Representation	4
2.4.1 Vector Databases	5
2.4.2 Knowledge Graphs	6
2.5 Natural Language Processing	9
2.5.1 Rule-based Approaches	9
2.5.2 Language Model-based Approaches	9
2.6 Large Language Models for Question Answering	10
2.7 Retrieval-Augmented Generation (RAG)	10
2.8 Multilingual and Mixed-Language Query Processing	11
2.9 Unstructured Data Extraction	11
2.10 Agentic Artificial Intelligence Systems	11
2.10.1 Multi-Agent Architectures	12
2.11 Current Travel Assistant Systems	12

2.11.1	Rome2Rio	12
2.11.2	Google Maps	13
2.11.3	MindTrip	13
2.11.4	GuideGeek	13
3	System Design	15
3.1	Overall System Architecture	15
3.2	Knowledge Graph Modeling	16
3.2.1	Graph Data Model	16
3.2.2	Public Transport Subgraph	16
3.2.3	Schema Integration and Design Considerations	16
3.3	Agent-Based System Design	17
3.3.1	Rationale for Agent-Based Design	17
3.3.2	Core Agent Responsibilities	18
3.4	Multilingual Query Processing Flow	18
3.5	Cypher Query Generation and Validation	18
3.6	Tools and Technologies	18
4	System Implementation	19
4.1	Development Environment and Setup	19
4.2	Data Collection and Preprocessing	19
4.2.1	Public Transportation Data Sources	19
4.2.2	Unstructured PDF Data Processing	19
4.3	Knowledge Graph Construction	19
4.3.1	Node and Relationship Creation	19
4.3.2	Data Cleaning and Deduplication	20
4.4	Agent Implementation	20
4.4.1	Entity Extraction Agent	20
4.4.2	Intent Classification Agent	20
4.4.3	Database Routing Agent	20
4.4.4	Cypher Generation Agent	20
4.4.5	Cypher Validation Agent	20
4.5	Integration with Neo4j	20
4.5.1	Database Configuration and Connection	20

4.5.2	Query Execution Workflow	21
4.6	User Interaction Interface	21
4.7	Error Handling and Clarification Mechanism	21
5	Evaluation and Results	22
5.1	Evaluation Objectives	22
5.2	Experimental Setup	22
5.3	Evaluation Metrics	22
5.3.1	Intent Classification Accuracy	22
5.3.2	Cypher Query Validity	22
5.3.3	Response Correctness	22
5.4	Test Scenarios and Use Cases	23
5.5	Experimental Results	23
5.6	Results Analysis	23
6	Discussion	24
6.1	Key Findings	24
6.2	Comparison with Existing Systems	24
6.3	Strengths of the Proposed Approach	24
6.4	Limitations and Challenges	24
6.5	Future Research Directions	24
	References	26

LIST OF FIGURES

Figure	Description	Page
Figure 2.1	Example Knowledge Graph	7
Figure 2.2	RAG Workflow	10
Figure 3.1	High Level Architecture	15

LIST OF TABLES

Table	Description	Page
Table 2.1	Comparison of Vector Databases	6
Table 2.2	Comparison of Knowledge Graph	8
Table 2.3	Comparison of Travel Guidance Platforms	14

CHAPTER 1

INTRODUCTION

1.1 Background

Transportation is central to Sri Lanka's social and economic development, supporting daily mobility, education, commerce, and regional growth. Public transportation, particularly buses and railways, provides affordable and widespread access[1]. An efficient transport network facilitates social connectivity, supports economic activities, and enables equitable access to essential services, making it a fundamental pillar of national development.

Despite its importance, navigating Sri Lanka's transportation system remains challenging for many users. Information related to routes, schedules, and fares is often difficult to access, particularly for individuals unfamiliar with local languages or regional geography. Transportation services are fragmented, and the absence of unified information systems makes travel planning complex and time-consuming[2]. This infrastructure constraints such as inadequate facilities for elderly passengers and individuals with disabilities restrict inclusive access to public transport, reducing overall system usability[3].

Fragmentation within the transportation network is further amplified by the coexistence of formal and informal transport services. Alongside state-operated buses and trains, a large informal sector including private buses and tuk-tuks operates without standardized fare structures or coordinated schedules[4]. While these services are essential, the lack of integration between formal and informal systems complicates route planning and fare estimation for users. This fragmented operational environment underscores the need for more integrated, user-oriented transportation solutions that improve accessibility, reliability, and ease of navigation across Sri Lanka's transport ecosystem.[5]

1.2 Motivation

Public bus transportation is the primary mode of travel for a large segment of the Sri Lankan population[6] however, access to reliable and accurate information regarding bus fares and schedules remains limited. Although official transport authorities publish this information, it is often distributed in static formats such as PDF documents that are difficult for the general public to search or interpret. Passengers are therefore required to rely on informal sources, manual inquiries, or incomplete digital platforms, which can lead to confusion and misinformation[7].

The motivation for this research arises from the need to bridge this gap by transforming fragmented public transport data into an accessible and intelligent information system[8]. By enabling natural-language interaction in Sinhala, Tamil, and English, the proposed system aims to reflect real-world user behavior and improve inclusivity. Structuring transport information into a knowledge-based system creates a foundation for interactive query handling[9], clarification of incomplete requests, and future extensions such as alternative route recommendations and intelligent transport services. This research is motivated by the need to improve public access to bus information and to support smarter, data-driven transportation solutions in Sri Lanka.

1.3 Problem Statement

Public bus fare and schedule information in Sri Lanka is fragmented and difficult to access[7], and existing systems fail to effectively support multilingual and mixed-language queries in Sinhala, Tamil, and English, such as “කොළඹේ මහනුවරට bus තියෙනවද?” or Colombo to Kandy fare eka kiyada?. Data inconsistencies, incomplete user queries, and the absence of structured multi-hop route recommendations when no direct bus service exists highlight the need for an intelligent, multilingual transport information system.

1.4 Research Gap

Current transportation platforms such as [GoogleMaps](#), [Rome2rio](#), [Moovit](#) and [MindTrip](#) provide basic route planning and recommendations but primarily support English and lack effective multilingual support, making them less accessible for Sinhala, Tamil queries and unable to offer language-aware route suggestions or alternative solutions.

1.5 Research Objectives

1. Develop a system capable of processing text-based prompts in Sinhala, Tamil, English and Singlish using a multilingual Agentic-LLM to deliver personalized travel guidance[10].
2. Develop a knowledge graph that aggregates fares, and routes, incorporating manual updates[11].
3. **Future Work:** Extend support for Chinese, and other languages to enhance accessibility.
4. **Future Work:** Implement speech-based input and output functionality, supporting interaction in multiple languages[12].

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

Natural Language Processing (NLP) is a field of artificial intelligence concerned with enabling computers to understand and generate human language. It encompasses a range of core tasks such as tokenization[13], part-of-speech tagging, named entity recognition, sentiment analysis[14], and machine translation, which together allow raw text to be transformed into structured and meaningful representations. Early NLP systems relied heavily on rule-based and statistical methods, whereas modern approaches increasingly use deep learning models to capture contextual and semantic information from large text corpora[15]. These advancements have significantly improved the ability of machines to handle unstructured and ambiguous language.

Recent progress in NLP has led to practical applications such as question answering, conversational agents, automated translation, and voice assistants, enabling more natural and intuitive humancomputer interaction[16]. In information retrieval and dialogue systems, NLP techniques are used to identify user intent, extract relevant entities, and generate appropriate responses. These capabilities are particularly important in domains where users communicate in informal or mixed-language forms, requiring robust language understanding and normalization before structured data can be queried[17]. As a result, NLP forms a foundational component of modern intelligent query systems that bridge natural language input and structured knowledge sources.

2.2 Transportation Information Systems

Transportation Information Systems are digital platforms designed to collect, process, manage, and disseminate transportation-related data to support planning, navigation, and decision making for travelers and operators. These systems typically integrate information such as routes, schedules, fares, vehicle locations, and service updates to provide users with timely and relevant travel guidance. By combining data management, communication networks, and user interfaces, they enable functions such as route search, timetable lookup, journey planning, and service monitoring[18]. Modern transportation information systems increasingly incorporate real-time data feeds, predictive analytics, and intelligent query processing to improve accuracy and responsiveness. Overall, they play a crucial role in enhancing the efficiency, accessibility, and reliability of public and private transport services.

2.3 Intelligent Transportation Systems

Intelligent Transportation Systems (ITS) use advanced computing, communication, and sensing technologies to improve the efficiency, safety, and usability of transportation networks. By integrating data from vehicles, infrastructure, and users, these systems support functions such as route planning, fare estimation, schedule management, traffic monitoring, and incident detection[19]. Continuous data exchange enables real-time decision making, dynamic routing, and better coordination of transport services, helping to reduce congestion and improve travel reliability. However, many existing ITS solutions rely on rigid and technical query interfaces, which limits accessibility and ease of use for everyday users. Overall, ITS aim to create more adaptive, responsive, and data-driven transportation services through the coordinated use of digital technologies.

2.4 Knowledge Representation

Knowledge storage technologies play a critical role in organizing, managing, and retrieving vast amounts of information in modern AI systems. These technologies can be broadly categorized into systems designed for unstructured data, which leverage vector-based representations, and systems focused on structured data, such as graph-based approaches. Vector-based systems excel in storing high-dimensional embeddings, enabling efficient similarity searches and semantic understanding, which are essential for applications like recommendation systems, contextual search, and natural language processing tasks [20]. On the other hand, graph-based systems capture relationships between entities, allowing for structured reasoning, complex query handling, and inference over interconnected data [21]. These technologies complement each other by addressing different aspects of knowledge representation: one prioritizing meaning and contextual relevance, the other emphasizing logical structure and relationships. Together, they support robust knowledge management frameworks that power applications ranging from semantic search and recommendation to advanced reasoning and decision-making systems.

The integration of hybrid knowledge systems that combine the strengths of these approaches is becoming increasingly popular. For instance, combining vector-based representations for contextual understanding with graph-based structures for logical reasoning creates powerful frameworks for multi-faceted knowledge management [22]. Recent advancements also focus on optimizing storage systems for low-latency access, scalability, and real-time updates, enabling their deployment in dynamic environments such as conversational agents, recommendation systems, and autonomous systems.

2.4.1 Vector Databases

Vector databases are specialized storage and retrieval systems designed to handle high-dimensional embeddings efficiently. These databases enable fast similarity searches by storing data as vector representations[23], making them essential for various AI and machine learning applications, including recommendation systems, semantic search, anomaly detection, and large-scale retrieval-augmented generation (RAG) models [24]. Unlike traditional relational databases that rely on structured indexing, vector databases use Approximate Nearest Neighbor (ANN) search techniques [25] such as hierarchical navigable small world (HNSW) graphs[26], product quantization (PQ)[27], and locality-sensitive hashing (LSH) to optimize query performance in massive datasets[28]. These techniques allow for rapid retrieval while balancing accuracy and computational cost.

One of the key strengths of vector databases is their ability to capture semantic relationships within unstructured data, such as text, images, and audio, through embedding-based representations. This capability is particularly beneficial in applications like natural language processing (NLP), where textual queries are mapped into dense vector spaces to retrieve semantically similar documents or responses. Additionally, vector databases support dynamic updates and real-time indexing, making them suitable for adaptive AI systems that require continuous learning and knowledge expansion[23]. A major challenge in deploying vector databases lies in optimizing storage efficiency and search speed for large-scale datasets. Techniques such as dimensionality reduction (e.g, Principal Component Analysis[29], Uniform Manifold Approximation and Projection[30]) and hybrid approaches that combine structured metadata filtering with vector-based retrieval enhance performance. Moreover, integrating vector databases with knowledge graphs and traditional databases enables hybrid retrieval strategies that leverage both structured relationships and semantic similarity[31].

Semantic search in a vector database maps text queries to high-dimensional embeddings, allowing retrieval based on meaning rather than exact keyword matches. For example, in English, if a user searches for "affordable hotels near the beach," the vector database can return results like "budget-friendly seaside resorts" or "low-cost beachfront stays." Similarly, in Sinhala, if a user queries "අඩු මිලට වෙරළ ආසන්න හෝටල්" (affordable hotels near the beach), the system can retrieve results like "සුන්දර මුහුදු තීරයේ ඉතාමත් අඩු මිලේ නවාතැන්" (quiet and budget-friendly beachfront accommodations). By leveraging embeddings, semantic search ensures that results match intent rather than just exact words.

TABLE 2.1: COMPARISON OF VECTOR DATABASES

Feature	Pinecone	Weaviate	FAISS
Scalability	High (Managed Cloud)	Moderate (Self-hosted & Cloud)	High (Local Deployment)
Indexing Speed	Real-time	Fast	Fast (GPU Accelerated)
Search Efficiency	High (Optimized for Approximate Nearest Neighbor Search)	High (Hybrid Search Supported)	Very High (Optimized for Large-scale Vector Search)
Ease of Use	Easy (Fully Managed API)	Moderate (Setup Required)	Complex (Manual Setup Required)
Best For	Production AI apps	AI-powered search	Large-scale similarity search

2.4.2 Knowledge Graphs

A Knowledge Graph (KG) is a structured representation of real-world entities and their relationships, often modeled as a graph where nodes represent entities (e.g, people, organizations, concepts), and edges capture semantic relationships between them. Unlike traditional relational databases, knowledge graphs are designed to facilitate reasoning, inference, and retrieval of information in a flexible, interconnected manner. They leverage ontologies and schema definitions to provide a semantic layer that enhances data integration and contextual understanding. These properties make KGs valuable in tasks such as question answering (QA), recommendation systems, entity linking, and knowledge-based inference.

A key advantage of knowledge graphs is their ability to incorporate heterogeneous, multi-source data, making them especially useful for knowledge-intensive applications like biomedical research, finance, and legal document analysis. Several construction methods exist, including rule-based approaches, machine learning-based entity extraction, and knowledge fusion from structured and unstructured data. In low-resource language settings, KGs help by structuring linguistic and domain-specific knowledge, improving cross-lingual retrieval and information synthesis. Recent advancements also integrate KGs with large language models (LLMs) to enhance factual grounding and reduce hallucinations in generative AI systems [21].

Evaluating knowledge graphs involves multiple aspects, including schema con-

sistency, knowledge completeness, and query performance. Techniques like graph embeddings and knowledge graph completion help improve KGs by predicting missing relationships or refining noisy data [32]. Challenges in large-scale KG deployment include scalability, data sparsity, and knowledge evolution, where dynamic updates are essential for maintaining relevance [33]. Future directions suggest combining vector search with structured KG reasoning to enhance retrieval and improve the interpretability of AI models

As an example, consider a knowledge graph that includes concepts related to "transportation" in both English and Sinhala. In the graph, the node for "Bus" might be connected to "Station" through a relationship labeled "arrives at." . These relationships help in understanding the connectivity between different concepts within the domain. A simple structure of a knowledge graph for transportation might look like this:

Bus (English) $\longrightarrow$ Arrives at $\longrightarrow$ Station (English).

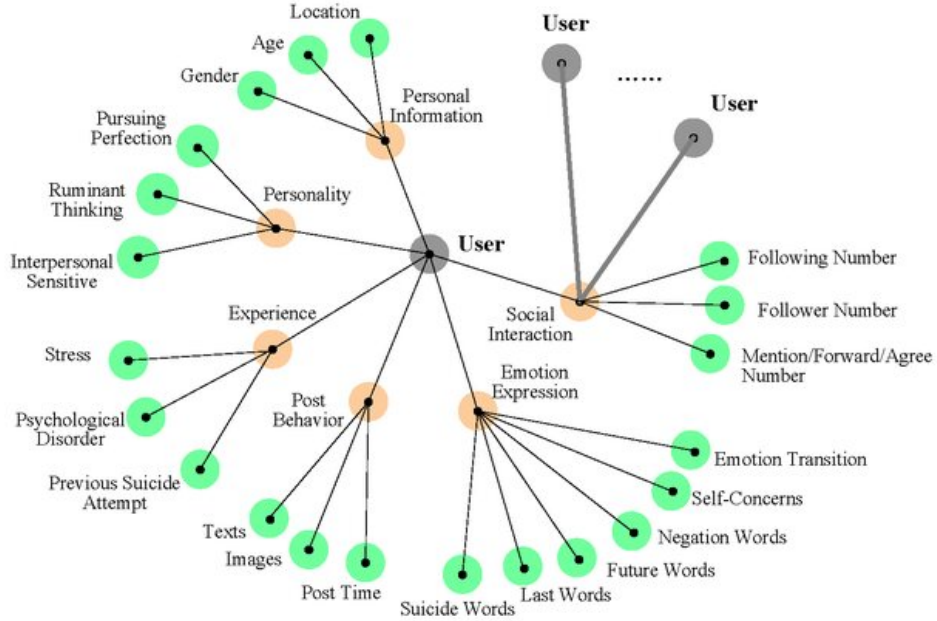


Fig. 2.1: Example Knowledge Graph

Note. For more details, visit [ResearchGate](#).

TABLE 2.2: COMPARISON OF KNOWLEDGE GRAPH

Technology	Functionality	Pros	Cons
Neo4j	Graph database with support for property graphs and Cypher query language.	Highly performant for graph traversals, mature ecosystem, good for real-time applications.	Complex scaling issues, may not be ideal for handling large amounts of unstructured data.
Apache Jena	Framework for building semantic web and linked data applications, supports RDF, SPARQL.	Well-suited for RDF graphs, open-source, integrates with Apache Hadoop and SPARQL endpoints.	Not optimized for high-performance graph processing or large-scale data.
RDFLib	Python library for working with RDF data and SPARQL queries.	Easy to use, good for small to medium-sized RDF data sets, integrates well with Python-based applications.	Limited scalability for very large datasets, no native graph database functionality.
Stardog	Enterprise-level graph database supporting both RDF and property graphs.	High scalability, strong support for reasoning and querying, rich set of features.	Expensive for large-scale implementations, complex configuration.
GraphDB	RDF graph database that supports SPARQL and provides advanced graph processing capabilities.	Excellent support for semantic web and linked data, highly scalable.	Performance may decrease with large, complex queries, complex setup for beginners.

2.5 Natural Language Processing

2.5.1 Rule-based Approaches

Rule-based approaches in Natural Language Processing (NLP) rely on manually defined linguistic patterns and grammatical rules to analyze and interpret text. These systems use predefined vocabularies, regular expressions, and syntactic rules to identify keywords, extract entities, and understand simple sentence structures[34]. For example, a rule-based query parser for transportation might match patterns like from X to Y or Sinhala equivalents to extract origin and destination locations from a user query. Such methods are transparent, predictable, and easy to control, making them useful for well-defined and domain-specific tasks.

However, rule-based NLP systems are limited in handling language variation, ambiguity, and mixed or informal expressions. They require continuous manual updates to cover new phrasing styles, spelling variations[35], or multilingual and code-mixed inputs (e.g., Sinhala-English Singlish). As a result, while rule-based approaches can provide high precision for constrained patterns, they struggle to generalize across diverse natural language queries compared to modern data-driven and learning-based NLP methods.

2.5.2 Language Model-based Approaches

Language modelbased approaches in Natural Language Processing (NLP) use statistical and deep learning models trained on large collections of text to understand and generate human language. Instead of relying on manually written rules, these models learn patterns of words, grammar, and meaning directly from data. Modern transformer-based language models can interpret user intent, handle variations in phrasing, and work across multiple languages or mixed-language inputs[36]. For example, a single model can understand queries such as Colombo to Kandy fare eka kiyada? or "කොළඹෙන් මහනුවරට bus තියෙනවද?" without explicitly programmed patterns for each format.

These approaches are more flexible and scalable than rule-based systems[37], as they can generalize to unseen queries and tolerate spelling mistakes, informal wording, and code-mixed language. They also enable advanced capabilities such as semantic search, question answering, and natural language to database query translation[38]. However, they typically require significant training data and computational resources, and their behaviour can be less transparent than rule-based methods. Despite these challenges, language modelbased NLP provides a powerful foundation for building natural, multilingual interfaces to complex data systems.

2.6 Large Language Models for Question Answering

Large Language Models (LLMs) have transformed question answering by using deep learning to understand and generate human-like responses. Trained on massive datasets, LLMs can handle complex queries, provide context-aware answers, and manage conversational follow-ups[39]. They can work alone or in combination with external knowledge sources through retrieval-augmented generation (RAG), improving accuracy by grounding answers in relevant data. Examples include GPT, Claude, and LLaMA, which differ in reasoning ability, multilingual support, and integration flexibility[40]. Compared to traditional rule-based or simpler language model approaches, LLMs offer more adaptive, interactive, and human-like responses, though they require substantial computational resources.

2.7 Retrieval-Augmented Generation (RAG)

RAG combines knowledge retrieval with language generation, where the model queries a database for real-time data (e.g. fare changes or delays) and generates responses [41]. This ensures up-to-date and accurate information delivery in tourism scenarios. There are two main types of RAG systems: End-to-End RAG, which integrates retrieval and generation into a seamless pipeline, optimizing for efficiency and cohesiveness [42], and Modular RAG, which separates these components, allowing for independent updates to the knowledge base and improvements to the language model [43]. Modular RAG is particularly beneficial in dynamic domains like transportation, as it enables frequent database updates without retraining the entire system.

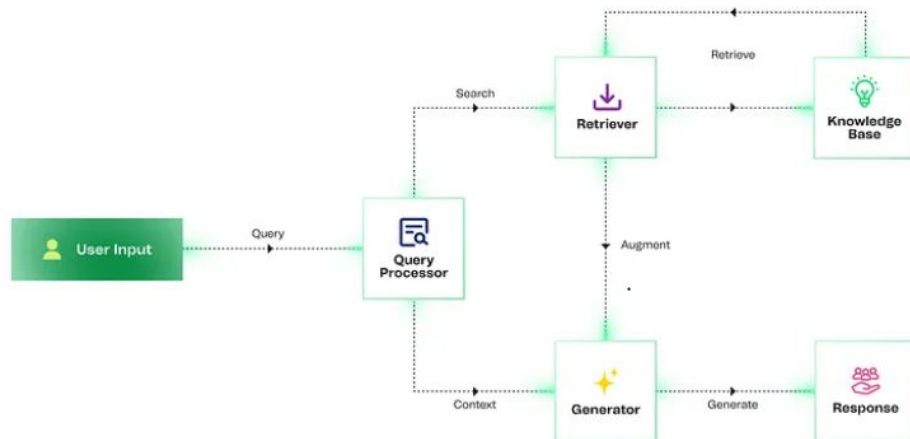


Fig. 2.2: RAG Workflow

Note. For more details, visit [Medium](#).

2.8 Multilingual and Mixed-Language Query Processing

Multilingual and mixed-language query processing enables systems to understand and respond to user queries expressed in different languages or a combination of languages. This involves detecting the language(s) used, translating or normalizing the input, and mapping it to a common representation for processing[44]. Such approaches improve accessibility and usability, allowing users to interact naturally without being restricted to a single language. Techniques often combine natural language processing, machine translation, and cross-lingual embeddings to handle language variations, code-switching, and context-sensitive meanings effectively. This capability is increasingly important in global and diverse environments where users may use multiple languages in the same query.

2.9 Unstructured Data Extraction

Unstructured data extraction refers to the process of converting information that is not organized in a predefined schema, such as text in documents, tables, and scanned images, into structured formats suitable for analysis or integration into databases. In the context of PDF documents, this task is particularly challenging because PDFs can store content as plain text, scanned images, or a combination of both, often with complex layouts[45]. Traditional approaches for PDF extraction include tools like PDFPlumber, PyMuPDF, and Tabula, which can parse textual content, extract tables, and identify page layouts[46]. For scanned documents or images embedded in PDFs, Optical Character Recognition (OCR) technologies such as Tesseract are widely used to convert visual text into machine-readable text. These methods require careful handling of issues like font variations, text alignment, and noise in scanned documents to ensure accurate extraction.

2.10 Agentic Artificial Intelligence Systems

Agentic Artificial Intelligence (AAI) systems are designed to perform autonomous tasks by perceiving their environment, reasoning over information, and taking goal-directed actions to achieve specific objectives[47]. Unlike traditional AI, which typically responds passively to inputs, agentic AI exhibits proactive behavior, planning sequences of actions, making decisions under uncertainty, and adapting to dynamic contexts. These systems often integrate multiple AI components, including natural language processing, knowledge representation, and machine learning, to operate as independent or semi-independent agents within complex workflows. Agentic AI finds applications in areas such as autonomous vehicles, intelligent virtual assistants, and multi-agent simulations, where continuous decision-making, coordination, and context-aware responses are critical[48]. By combining perception, reasoning, and action, agentic AI systems aim to create more intelligent, flexible, and self-directed solutions capable of addressing

real-world problems efficiently.

2.10.1 Multi-Agent Architectures

Multi-agent architectures consist of a system of interacting autonomous agents that collaborate or compete to achieve individual or collective goals[49]. Each agent operates with its own knowledge, reasoning capabilities, and objectives, while communication and coordination protocols enable them to share information, negotiate, and synchronize actions effectively. These architectures are particularly useful for solving complex, distributed, or dynamic problems that are difficult for a single agent to handle, such as traffic management, logistics optimization, and intelligent transportation systems[50]. By leveraging task decomposition, role assignment, and inter-agent collaboration, multi-agent systems can improve scalability, adaptability, and robustness. Additionally, such architectures allow for modular system design, where agents can be added, removed, or specialized without disrupting the overall system, making them highly flexible for evolving real-world applications. Popular frameworks, such as Langgraph, AutoGen, CrewAI and Haystack, offer built-in tools for agent coordination, task delegation, and monitoring, allowing developers to design modular, scalable, and flexible multi-agent systems[51].

2.11 Current Travel Assistant Systems

Modern travel and navigation platforms such as Google Maps, Trip.com, and Rome2Rio provide users with essential travel information, including route planning, public transport options, real-time traffic updates, and accommodation recommendations. These systems integrate various data sources and employ AI-driven algorithms to enhance the user experience. However, despite their effectiveness, they have several limitations.

2.11.1 Rome2Rio

Rome2Rio is a comprehensive travel planning platform that helps users find the best transportation routes between two locations, combining options like flights, trains, buses, ferries, and driving. With a user-friendly interface and global coverage, it allows travelers to compare routes, travel times, and costs across various modes of transportation, making it easy to plan both domestic and international trips. Although it offers booking integration with some providers, it doesn't handle all bookings directly and may lack real-time updates for certain routes. Nonetheless, it remains a valuable tool for travelers seeking to explore efficient and cost-effective travel options worldwide.

This offer multi-transport options (bus, train, flight), they often lack real-time updates for every mode and low resource language support. The accuracy of schedule information for local or less popular transport networks remains a challenge, leading to incorrect travel planning [52]. [Rome2Rio](#)

2.11.2 Google Maps

Google Maps is a widely used navigation and mapping service that provides users with detailed maps, real-time traffic information, and directions for driving, walking, cycling, and public transportation. With its global coverage and user-friendly interface, it helps users find the quickest and most efficient routes, offering real-time updates on traffic conditions, accidents, and road closures. Provides real-time traffic and transit updates, but its reliability depends on data availability from transit agencies and GPS tracking. Rural or developing regions with low digital infrastructure often experience outdated or missing information [53]. [Google Maps](#)

2.11.3 MindTrip

Mindtrip is an innovative travel platform designed to provide personalized trip planning, offering tailored itineraries and experiences based on individual preferences, interests, and budgets. With a user-friendly interface, Mindtrip helps travelers discover unique destinations, activities, and accommodations, while also integrating real-time recommendations and local insights. This prioritizes sponsored listings, potentially limiting the visibility of more budget-friendly or local options. Real-time availability and pricing updates may not always be accurate, leading to discrepancies in bookings and not updated in Sri lankan context. [MindTrip](#)

2.11.4 GuideGeek

GuideGeek AI is a chatbot-based travel assistant that integrates with social media platforms like WhatsApp for real-time travel guidance. It supports multiple languages and provides conversational AI assistance. However, its functionality is primarily focused on chat-based interactions, with limited standalone app features. [GuideGeek](#)

TABLE 2.3: COMPARISON OF TRAVEL GUIDANCE PLATFORMS

Platform	Developed By	Technologies Used	Supported Languages	Pros & Cons
Mindtrip	Mindtrip AI	AI, NLP, LLM	English (Multilingual planned)	+ AI-powered personalized planning - Limited language support
Google Maps	Google	AI, ML, Crowdsourcing, GIS	50+ Languages	+ Extensive global coverage - Privacy concerns
Rome2Rio	Rome2Rio Pty Ltd.	API Integration, Graph Search	English (Limited multilingual support)	+ Multi-modal travel options - Limited real-time updates
TripIt	SAP Concur	Email Parsing, NLP	Multiple languages	+ Easy itinerary management - No direct booking
TravelPerk	TravelPerk Ltd.	Cloud Computing, AI	English, Spanish, French, German	+ Business travel-focused - Primarily for corporate users
Vacay	Vacay AI	AI, Recommender Systems	Multilingual	+ Personalized recommendations - Limited user base
GuideGeek	GuideGeek AI	AI, Chatbot, Social Media Integration	Multiple languages	+ Conversational AI, WhatsApp integration - Limited app functionality

CHAPTER 3

SYSTEM DESIGN

3.1 Overall System Architecture

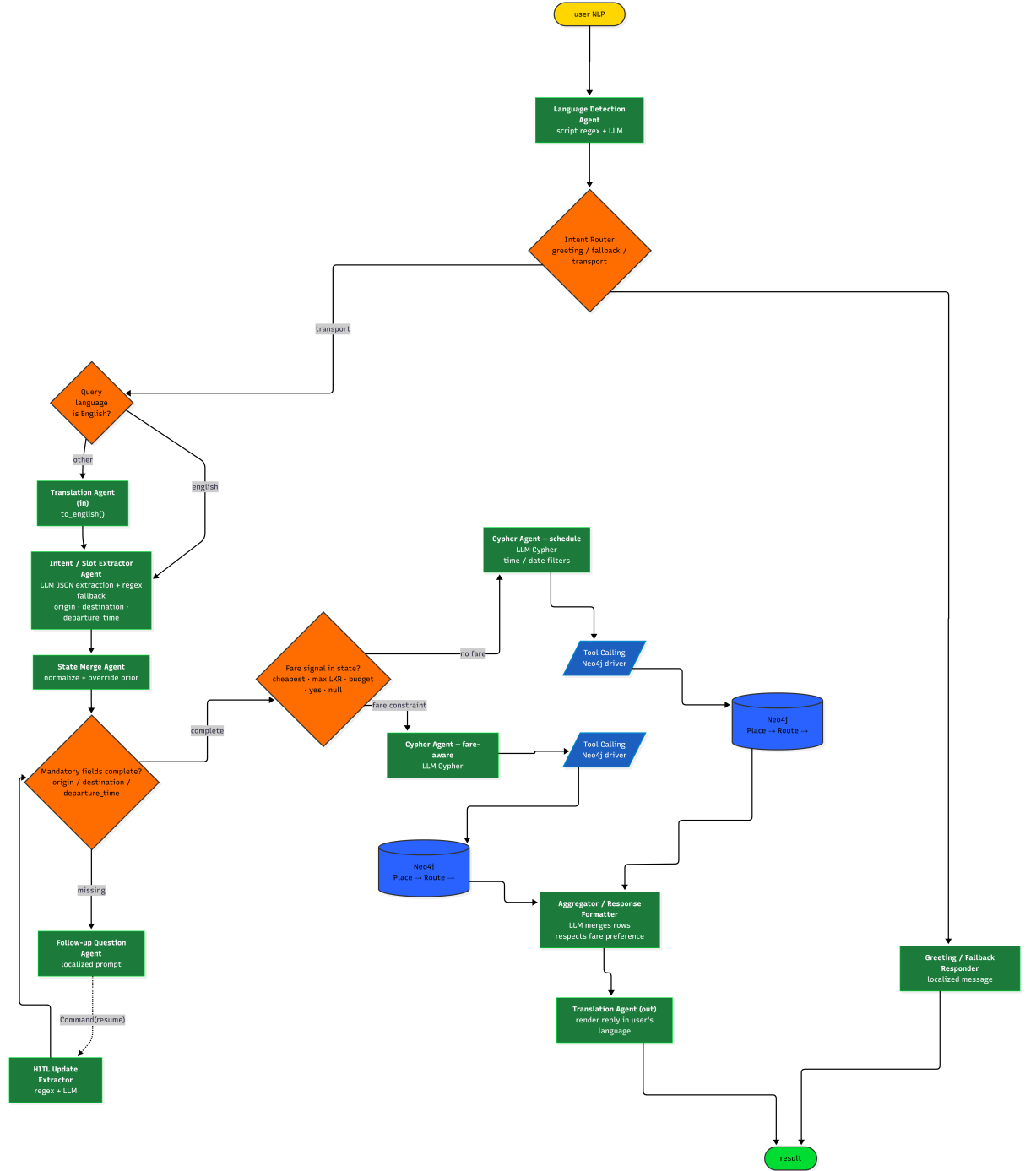


Fig. 3.1: High Level Architecture

3.2 Knowledge Graph Modeling

The Knowledge Graph (KG) is designed to provide a structured and semantically enriched representation of public and private transport information in Sri Lanka. The system adopts a labeled property graph model implemented using Neo4j AuraDB, where domain entities are represented as nodes, their interactions as relationships, and descriptive attributes as properties. This modeling approach enables efficient graph traversal, expressive Cypher querying, and semantic reasoning over transport data.

3.2.1 Graph Data Model

The knowledge graph consists of three primary components:

- **Nodes:** Core entities such as `Place`, `Route`, `Trip`, `Stop`, `Operator`, `Driver`, and `Vehicle`.
- **Relationships:** Semantic connections including `CONNECTS_TO`, `HAS_FARE`, `HAS_SCHEDULE`, `OPERATED_BY`, and `DEPARTS_FROM`.
- **Properties:** Attributes such as departure time, arrival time, fare amount, service date, vehicle type, and capacity.

All temporal attributes are standardized using consistent time formats to support time-based filtering and comparison queries.

3.2.2 Public Transport Subgraph

The public transport subgraph models scheduled bus services extracted from official timetable PDFs and fare tables. `Place` nodes represent cities and major stops, while relationships such as `HAS_SCHEDULE` and `HAS_FARE` store temporal and pricing information between origin and destination pairs. Location name normalization and entity resolution techniques are applied to prevent duplication and ensure data consistency.

3.2.3 Schema Integration and Design Considerations

The graph design emphasizes scalability, modularity, query efficiency through indexed properties, and extensibility to support additional transport modes in future expansions. This unified knowledge representation serves as the semantic backbone of the agentic conversational system, enabling accurate and explainable graph-based information retrieval.

3.3 Agent-Based System Design

3.3.1 Rationale for Agent-Based Design

The proposed system adopts an agent-based architecture to address the inherent complexity of multilingual natural language understanding, dynamic query generation, and graph-based transport information retrieval. Traditional monolithic chatbot designs tightly couple language processing, business logic, and database interaction, resulting in limited scalability, reduced explainability, and difficulty in extending functionality. In contrast, an agent-based design decomposes the system into modular, task-specialized components that collaborate to achieve end-to-end conversational reasoning[54].

The transport guidance problem involves multiple cognitive steps, including language detection, intent classification, entity extraction, query formulation, database routing, and response synthesis. Each of these tasks requires distinct reasoning capabilities and may involve different prompt strategies or tool integrations[55]. By assigning these responsibilities to dedicated agents (e.g Intent Classification Agent, NL-to-Cypher Agent, Retrieval Agent, and Response Agent), the system achieves functional separation, improved maintainability, and clearer reasoning pathways.

The integration of LLMs benefits significantly from agent orchestration. Rather than relying on a single prompt to perform all reasoning steps, the agent-based approach enables structured decomposition of tasks, intermediate validation, and dynamic routing. This improves robustness, reduces hallucination risks in database query generation, and enhances controllability when interacting with the knowledge graph.

Another key motivation for the agent-based design is scalability and extensibility. Additional capabilities, such as real-time private hire integration or support for new transport modes, can be incorporated by introducing new specialized agents without redesigning the entire system architecture. This modular structure also supports experimentation, evaluation, and iterative refinement of individual agents[56].

Overall, the agent-based paradigm provides a flexible, explainable, and scalable framework for integrating multilingual natural language interfaces with graph-based knowledge systems, making it well-suited for complex, real-world transport information environments.

3.3.2 Core Agent Responsibilities

Key agents are responsible for entity extraction, intent classification, query routing, and Cypher generation, each operating with clearly defined input and output constraints.

3.4 Multilingual Query Processing Flow

Natural language queries are transformed into structured representations through entity and intent extraction. An information completeness check ensures that insufficient queries trigger clarification rather than incorrect query generation.

3.5 Cypher Query Generation and Validation

Cypher queries are generated using predefined templates aligned with the knowledge graph schema. Schema validation mechanisms are applied to prevent hallucinated graph structures and ensure query correctness.

3.6 Tools and Technologies

The system utilizes Neo4j for knowledge graph storage, CrewAI for agent orchestration, and large language models for natural language understanding and query generation.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Development Environment and Setup

The system was implemented using Python as the primary programming language. The development environment consisted of a local workstation configured with Neo4j as the graph database and CrewAI for agent orchestration. Version control was managed using Git to support iterative development and experimentation.

4.2 Data Collection and Preprocessing

Transportation-related data was collected from multiple sources and processed to ensure consistency and usability within the knowledge graph.

4.2.1 Public Transportation Data Sources

Public transportation data, including fare information and service schedules, was obtained from officially published sources. These datasets provided structured information on routes, locations, and pricing.

4.2.2 Unstructured PDF Data Processing

Schedule data available in unstructured PDF format was manually inspected and converted into structured representations. Data fields such as route names, departure times, and destinations were extracted and standardized prior to graph insertion.

4.3 Knowledge Graph Construction

The processed data was modeled and stored using a property graph representation to support efficient traversal and querying.

4.3.1 Node and Relationship Creation

Graph nodes representing transportation locations were created, along with relationships capturing fare and schedule connections. Relevant attributes were stored as relationship properties to enable expressive queries.

4.3.2 Data Cleaning and Deduplication

Data cleaning techniques were applied to remove duplicate entities and resolve inconsistencies in naming conventions. This step ensured accurate graph connectivity and reduced redundancy.

4.4 Agent Implementation

The agentic query processing layer was implemented using the CrewAI framework, with each agent designed to perform a specific task within the query pipeline.

4.4.1 Entity Extraction Agent

This agent is responsible for extracting key entities such as origin, destination, date, and time from user queries and converting them into a structured format.

4.4.2 Intent Classification Agent

The intent classification agent determines whether a user query relates to fare information, schedule information, both, or requires clarification.

4.4.3 Database Routing Agent

Based on the classified intent, this agent decides which knowledge graph database should be queried to retrieve relevant information.

4.4.4 Cypher Generation Agent

The Cypher generation agent translates structured query representations into schema-constrained Cypher queries using predefined templates.

4.4.5 Cypher Validation Agent

This agent validates generated Cypher queries to ensure compliance with the knowledge graph schema and prevent invalid or unsafe queries.

4.5 Integration with Neo4j

The agentic system was integrated with Neo4j to enable real-time execution of generated Cypher queries.

4.5.1 Database Configuration and Connection

Neo4j was configured as the backend graph database, and secure connections were established using the official Neo4j Python driver.

4.5.2 Query Execution Workflow

Validated Cypher queries were executed against the appropriate database, and results were retrieved in a structured format for further processing.

4.6 User Interaction Interface

A chat-based user interface was implemented to allow users to submit natural language transportation queries and receive human-readable responses.

4.7 Error Handling and Clarification Mechanism

The system incorporates error handling mechanisms to manage incomplete or ambiguous user queries. When necessary, clarification prompts are generated to request additional information from the user before query execution.

CHAPTER 5

EVALUATION AND RESULTS

5.1 Evaluation Objectives

The evaluation aims to assess the effectiveness of the proposed agentic system in translating natural language transportation queries into accurate and executable Cypher queries. The primary objectives include measuring intent classification accuracy, validating generated Cypher queries, and assessing the correctness of final system responses.

5.2 Experimental Setup

Experiments were conducted using a set of predefined transportation-related queries covering fare inquiries, schedule inquiries, and combined requests. The system was tested on the constructed knowledge graphs, and all agents were executed sequentially under identical conditions to ensure consistent results.

5.3 Evaluation Metrics

Quantitative and qualitative metrics were employed to evaluate system performance at different stages of the query processing pipeline.

5.3.1 Intent Classification Accuracy

Intent classification accuracy measures the proportion of user queries that were correctly categorized into predefined intent classes. This metric evaluates the systems ability to correctly interpret user requirements.

5.3.2 Cypher Query Validity

Cypher query validity assesses whether generated queries conform to the knowledge graph schema and execute successfully without errors. Invalid or hallucinated queries were recorded and analyzed.

5.3.3 Response Correctness

Response correctness evaluates the accuracy and relevance of the systems final responses when compared against expected results derived from the knowledge graph data.

5.4 Test Scenarios and Use Cases

Test scenarios were designed to represent realistic user interactions, including incomplete queries, ambiguous location names, and combined fare and schedule requests. These scenarios were used to assess system robustness and error-handling capabilities.

5.5 Experimental Results

The experimental results demonstrate that the agentic approach effectively supports accurate intent classification and reliable Cypher query generation. Most queries were successfully processed end-to-end, yielding correct and contextually appropriate responses.

5.6 Results Analysis

Analysis of the results indicates that the use of specialized agents improves system reliability and reduces query generation errors. Misclassifications and invalid queries primarily occurred in cases of highly ambiguous or incomplete user input, highlighting areas for future improvement.

CHAPTER 6

DISCUSSION

6.1 Key Findings

The findings of this research indicate that an agentic architecture is effective for translating natural language transportation queries into structured graph queries. The modular separation of responsibilities among agents improved the systems interpretability and reduced the likelihood of invalid query generation. The results demonstrate that the use of intermediate structured representations enhances overall system reliability.

6.2 Comparison with Existing Systems

Compared to traditional rule-based natural language interfaces and end-to-end LLM-driven approaches, the proposed system offers improved transparency and control. Unlike monolithic systems that directly map user input to database queries, the agentic approach enables incremental reasoning and validation at each stage, reducing schema hallucination and execution errors.

6.3 Strengths of the Proposed Approach

A key strength of the proposed approach is its modular design, which allows individual agents to be refined or replaced without affecting the entire system. The separation of fare and schedule knowledge graphs further enhances scalability and supports flexible query routing. Additionally, the use of schema-constrained Cypher generation improves safety and query correctness.

6.4 Limitations and Challenges

Despite its advantages, the system faces challenges related to ambiguous user input and variations in natural language expression. Performance is also dependent on the quality of the underlying knowledge graph and the accuracy of entity normalization. Furthermore, the reliance on external language models introduces latency and potential cost considerations.

6.5 Future Research Directions

Future work may extend the system by incorporating additional transportation modes and real-time data sources to enhance coverage and responsiveness. Further research could explore adaptive learning mechanisms to improve entity recognition and intent classification over time. Additionally, integrating explainable

AI techniques and user feedback loops may further improve system usability and trustworthiness.

REFERENCES

- [1] A. Kumarage, K. Gwilliams, and J. DS, “Developing public transport in sri lanka,” 01 2005.
- [2] P. O’Sullivan and T. Patel, “Fragmentation in transport operations and the case for system integrity,” *Transport Policy*, vol. 11, pp. 215–225, 02 2004.
- [3] V. Tennakoon, J. Wiles, R. Peiris-John, R. Wickremasinghe, B. Kool, and S. Ameratunga, “Transport equity in sri lanka: Experiences linked to disability and older age,” *Journal of Transport & Health*, vol. 18, p. 100913, 2020.
- [4] R. Cervero and A. Golub, “Informal public transport: A global perspective,” *Urban Transport in the Developing World: A Handbook of Policy and Practice*, pp. 488–518, 01 2011.
- [5] R. Ranasinghe and T. Sivakumar, “Optimising multimodal integration: Global best practices for seamless connectivity and their applicability in sri lanka,” *Journal of South Asian Logistics and Transport*, vol. 5, pp. 97–116, 09 2025.
- [6] N. Fernando, S. Karunarathna, B. Wanasooriya, P. Baddewela, S. Thillaiampalam, and T. Liyanage, “Bus transport service in sri lanka: A perception of users,” 08 2023.
- [7] A. Kumarage, “Review of transport sector in sri lanka,” 07 2010.
- [8] A. F. Nuskiya, “Impact of digital marketing in tourism industry of sri lanka,” in *Information and Knowledge Management*, vol. 8, no. 9, 2018, pp. 59–67.
- [9] K. Chanumolu, “Structuring the intelligent transport systems based on internet of things (iot),” *International Journal of Engineering Trends and Technology*, vol. 35, pp. 525–531, 05 2016.
- [10] M. Á. Álvarez-Carmona, R. Aranda, A. Y. Rodríguez-Gonzalez, D. Fajardo-Delgado, M. G. Sánchez, H. Pérez-Espinoza, J. Martínez-Miranda, R. Guerrero-Rodríguez, L. Bustio-Martínez, and Á. Díaz-Pacheco, “Natural language processing applied to tourism research: A systematic review and future research directions,” *Journal of king Saud university-computer and information sciences*, vol. 34, no. 10, pp. 10 125–10 144, 2022.
- [11] A. Munawar, S. A. Raza, and A. Qasim, “Design and development of ai-based tourist facilitator and information agent,” *Applied Computer Systems*, vol. 25, no. 2, pp. 124–133, 2020.

- [12] B. Sougato, H. M. Rakibul, M. S. M. A. Sk, and M. Jobayer, “Automating transport inquiries: A voice-activated solution for modern travel needs.”
- [13] S. J. Mielke, Z. Alyafeai, E. Salesky, C. Raffel, M. Dey, M. Gallé, A. Raja, C. Si, W. Y. Lee, B. Sagot *et al.*, “Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp,” *arXiv preprint arXiv:2112.10508*, 2021.
- [14] M. T. Khan, M. Durrani, A. Ali, I. Inayat, S. Khalid, and K. H. Khan, “Sentiment analysis and the complex natural language,” *Complex Adaptive Systems Modeling*, vol. 4, pp. 1–19, 2016.
- [15] A. James, P. Herny, and A. Samuel, “The evolution of nlp: A journey through time,” 05 2025.
- [16] Q. Xu, “Practical applications of large language models in enterprise-level applications,” *Journal of Computer Science and Artificial Intelligence*, vol. 2, pp. 17–21, 02 2025.
- [17] L. Gonzalez-Gomez, S. Hernandez-Munoz, A. Borja, J. Azofeifa, J. Noguez, and P. Caratozzolo, “Analyzing natural language processing techniques to extract meaningful information on skills acquisition from textual content,” *IEEE Access*, vol. PP, pp. 1–1, 01 2024.
- [18] J. Zapata, M. Arango-Serna, and R. GOMEZ, “Information systems applied to transport improvement,” *DYNA*, vol. 80, pp. 77–86, 08 2013.
- [19] K. Shaaban, M. Elamin, and M. Alsoub, “Intelligent transportation systems in a developing country: Benefits and challenges of implementation,” *Transportation Research Procedia*, vol. 55, pp. 1373–1380, 01 2021.
- [20] S. S. Monir, I. Lau, S. Yang, and D. Zhao, “Vectorsearch: Enhancing document retrieval with semantic embeddings and optimized search,” *arXiv preprint arXiv:2409.17383*, 2024.
- [21] S. Ji, S. Pan, E. Cambria, P. Marttinen, and S. Y. Philip, “A survey on knowledge graphs: Representation, acquisition, and applications,” *IEEE transactions on neural networks and learning systems*, vol. 33, no. 2, pp. 494–514, 2021.
- [22] J. Mohoney, A. Pacaci, S. R. Chowdhury, A. Mousavi, I. F. Ilyas, U. F. Minhas, J. Pound, and T. Rekatsinas, “High-throughput vector similarity search in knowledge graphs,” *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–25, 2023.
- [23] J. J. Pan, J. Wang, and G. Li, “Survey of vector database management systems,” *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024.

- [24] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, “Improving the domain adaptation of retrieval augmented generation (rag) models for open domain question answering,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1–17, 2023.
- [25] C.-Y. Chiu, A. Prayoonwong, and Y.-C. Liao, “Learning to index for nearest neighbor search,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 8, pp. 1942–1956, 2019.
- [26] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [27] H. Jegou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 33, no. 1, pp. 117–128, 2010.
- [28] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng, “Query-aware locality-sensitive hashing for approximate nearest neighbor search,” *Proceedings of the VLDB Endowment*, vol. 9, no. 1, pp. 1–12, 2015.
- [29] H. Abdi and L. J. Williams, “Principal component analysis,” *Wiley interdisciplinary reviews: computational statistics*, vol. 2, no. 4, pp. 433–459, 2010.
- [30] L. McInnes, J. Healy, and J. Melville, “Umap: Uniform manifold approximation and projection for dimension reduction,” *arXiv preprint arXiv:1802.03426*, 2018.
- [31] C. Wei, B. Wu, S. Wang, R. Lou, C. Zhan, F. Li, and Y. Cai, “Analyticdbv: a hybrid analytical engine towards query fusion for structured and unstructured data,” *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3152–3165, 2020.
- [32] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich, “A review of relational machine learning for knowledge graphs,” *Proceedings of the IEEE*, vol. 104, no. 1, pp. 11–33, 2015.
- [33] H. Paulheim, “Knowledge graph refinement: A survey of approaches and evaluation methods,” *Semantic web*, vol. 8, no. 3, pp. 489–508, 2016.
- [34] T. Mahmud, K. M. Azharul Hasan, M. Ahmed, and T. H. C. Chak, “A rule based approach for nlp based query processing,” in *2015 2nd International Conference on Electrical Information and Communication Technologies (EICT)*, 2015, pp. 78–82.

- [35] A. Salama, H. E. Khalid, E. Essa, and N. Ahmed, “A natural language processing environment for rule- based decision making with neutrosophic logic to manage uncertainty and ambiguity,” p. 2025, 03 2025.
- [36] P. Banerjee and H. Han, “Language modeling approaches to information retrieval,” *JCSE*, vol. 3, pp. 143–164, 09 2009.
- [37] S. Yeung, “A comparative study of rule-based, machine learning and large language model approaches in automated writing evaluation (awe),” 03 2025, pp. 984–991.
- [38] N. Sharef, “Natural language query translation for semantic search,” *International Journal of Digital Content Technology and its Applications*, vol. 7, pp. 53–63, 10 2013.
- [39] A. Matarazzo and R. Torlone, “A survey on large language models with some insights on their capabilities and limitations,” 01 2025.
- [40] A. Gupta, “A comparative analysis of large language models for business applications,” 07 2025.
- [41] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [42] H. Zamani and M. Bendersky, “Stochastic rag: End-to-end retrieval-augmented generation through expected utility maximization,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 2641–2646.
- [43] J. Jin, Y. Zhu, X. Yang, C. Zhang, and Z. Dou, “Flashrag: A modular toolkit for efficient retrieval-augmented generation research,” *arXiv preprint arXiv:2405.13576*, 2024.
- [44] M. Lui, J. Lau, and T. Baldwin, “Automatic detection and language identification of multilingual documents,” *Transactions of the Association for Computational Linguistics*, vol. 2, pp. 27–40, 12 2014.
- [45] A. Hashmi and M. Afzal, “Insights to the state-of-the-art pdf extraction techniques,” 06 2020.
- [46] N. Adhikari and S. Agarwal, “A comparative study of pdf parsing tools across diverse document categories,” 10 2024.
- [47] D. B. Acharya, K. Kuppan, and B. Divya, “Agentic ai: Autonomous intelligence for complex goalsa comprehensive survey,” *IEEE Access*, vol. 13, pp. 18 912–18 936, 2025.

- [48] O. Khalid, A. Farooqi, and M. Bilal, “Agentic ai: A review, applications, and open research challenges,” 12 2025.
- [49] O. Khozium, “Multi-agent system overview: Architectural designing using practical approach,” *Journal of Computer Science and Technology*, vol. 5, 05 2013.
- [50] I. Abbasnejad, X. Liu, and A. Roy, “Deciding the path: Leveraging multi-agent systems for solving complex tasks,” 06 2025, pp. 4216–4225.
- [51] A. Ramachandran, “A survey of agentic ai, multi-agent systems, and multimodal frameworks architectures, applications, and future directions,” 12 2024.
- [52] C. Llorca, J. Molloy, J. Ji, and R. Moeckel, “Estimation of a long-distance travel demand model using trip surveys, location-based big data, and trip planning services,” *Transportation Research Record*, vol. 2672, no. 47, pp. 103–113, 2018.
- [53] A. A. Abdulrahman, “Google maps problems in iraq.”
- [54] S. Pan and D. Wu, “Modular task decomposition and dynamic collaboration in multi-agent systems driven by large language models,” 11 2025.
- [55] S.-Y. Huang, “Toward agentic ai: Task-oriented communication for hierarchical planning of long-horizon tasks,” 01 2026.
- [56] K. Yuksel and H. Sawaf, “A multi-ai agent system for autonomous optimization of agentic ai solutions via iterative refinement and llm-driven feedback loops,” 12 2024.